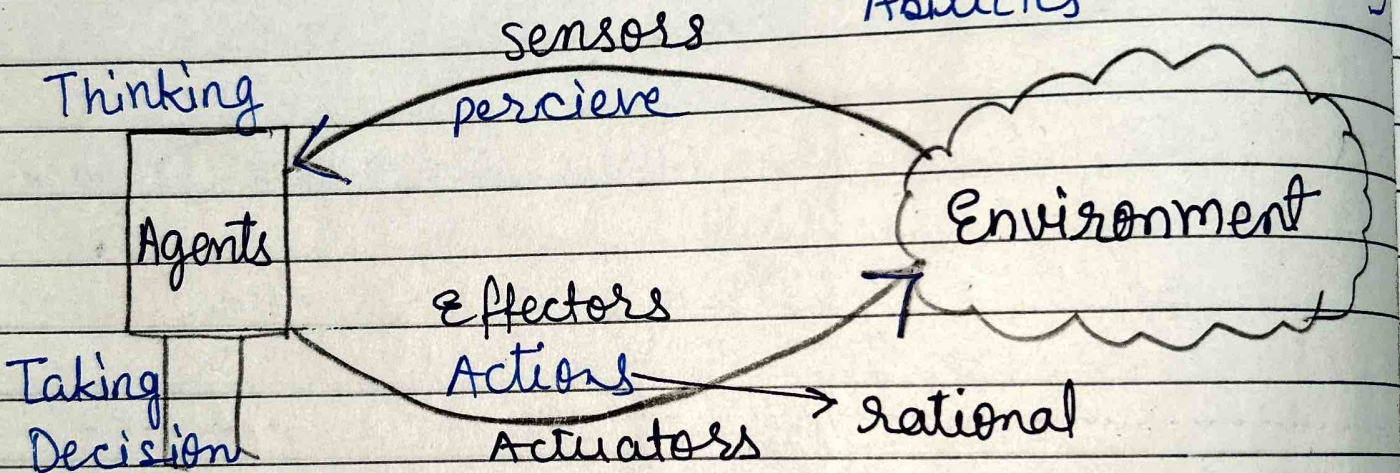
Intelligent AGENTS IN AI

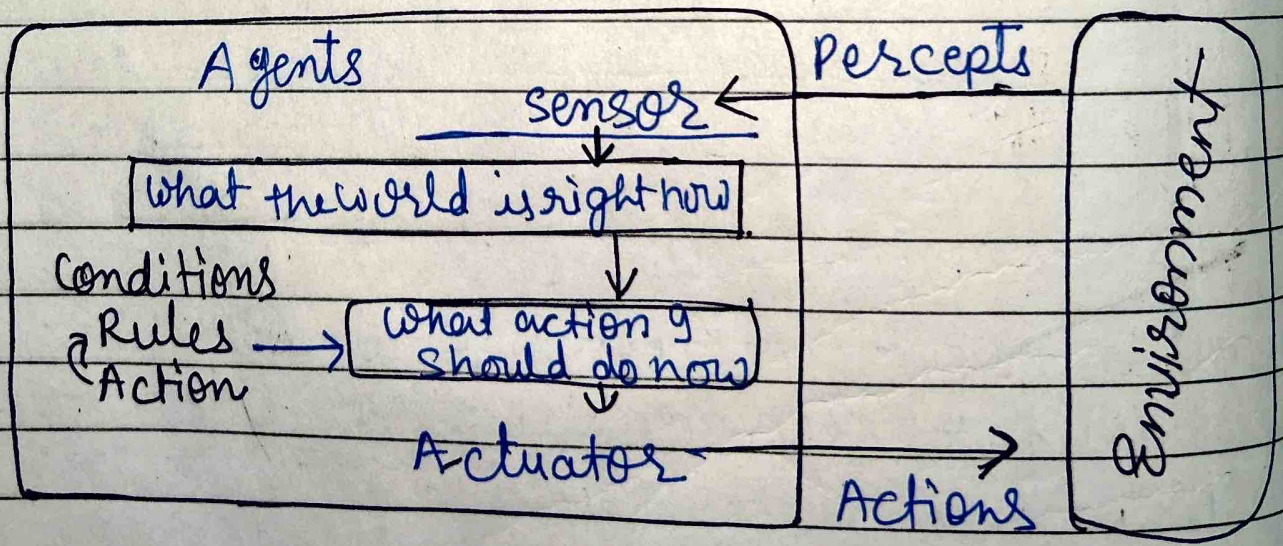
Intl (or percept)

"The agents sense the environment through sensors and act on their environment through actuatorss. An AI agent can have mental properties such as knowledge, belief, intention etc."

An agent runs in the cycle of perceiving, thinking and acting"

It takes action based on → Prior knowledge, Observation, Past Experience, Goal/preference, Abilities





Ex of Agents - Thermostat, Cellphone, Camera & human, Wrist band

Goal of Agents :- - High performance
- Optimal Result
- Rational Action (Right or correct)

PEAS :- It a type of model on which AI agent works upon

Ex - Self driving car (automated)

P (Performance) → Safety, time, legal, comfort

E (Environment) → other vehical/humans on Raads

A (Actuators) → Stearing, accelerator, brake, signal, horn,

S (Sensors) → Camera, GPS, speedometer

AGENTS = Architecture (Machinary that an agent execute on) + Agent Program (Implementation of an agent function)

Agent function is used to map a percept to an action

Sensors :- It is a device which detects the chang in the environment & sends the information to other electronic device. An agent observes its environment threrigh sensors.

Actuators :- Actuators are the component of machi that converts energy into motion. The actuators are only responsible for moving a controlling a system. Ex - electric motor, gears, ra

Effectors :- Effectors are the devices which affect

the environment. Ex- legs, wheels, arms, fingers, fins, & display screen.

Rational Agents:- A rational agents is said to perform the right things. AI is about creating rational agents to use for game theory & decision theory for various real world scenarios.

Rational agents → clear preference
→ Models uncertainty
→ High performance
→ All right possible action

Ex- Vacuum Cleaner (An agent) → (Simple reflex)
→ performance - cleanness, Efficiency, Battery life, security.
→ Environment - Room, table, wood floor, carpet
→ Actuator - Wheels, Brushes, Vacuum Extractor
→ Sesors - Camera, Dirt detection sesor, Infrared wall sensor

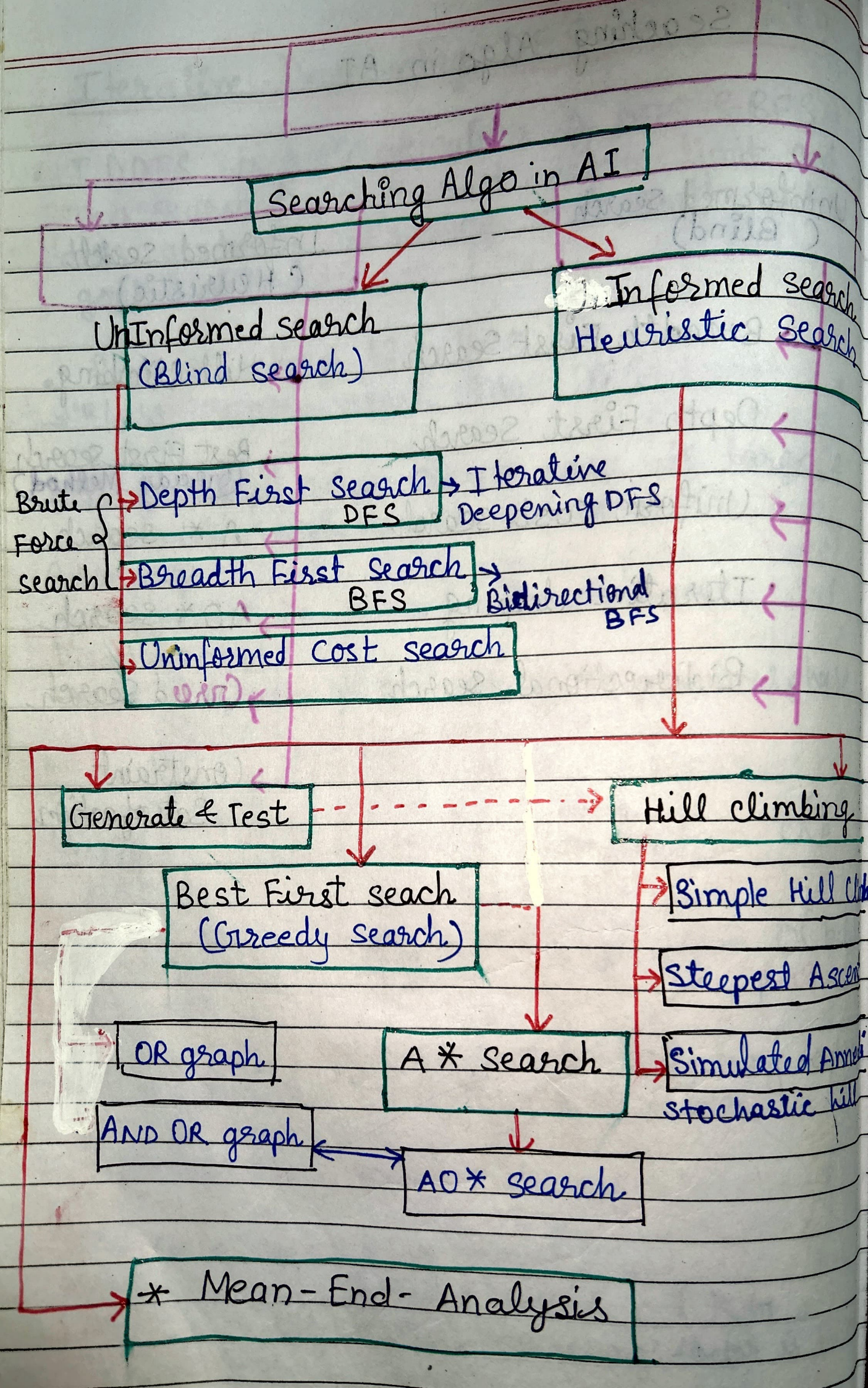
-: TYPES OF AI AGENTS :-

Agents can be grouped into five classes based on their degree of perceived intelligence and capability. All these agent can improve performance and generate better action over the time.

1) Simple Reflex agent
2) Model based reflex
3) Goal based agents
4) Utility based
5. Learning agents

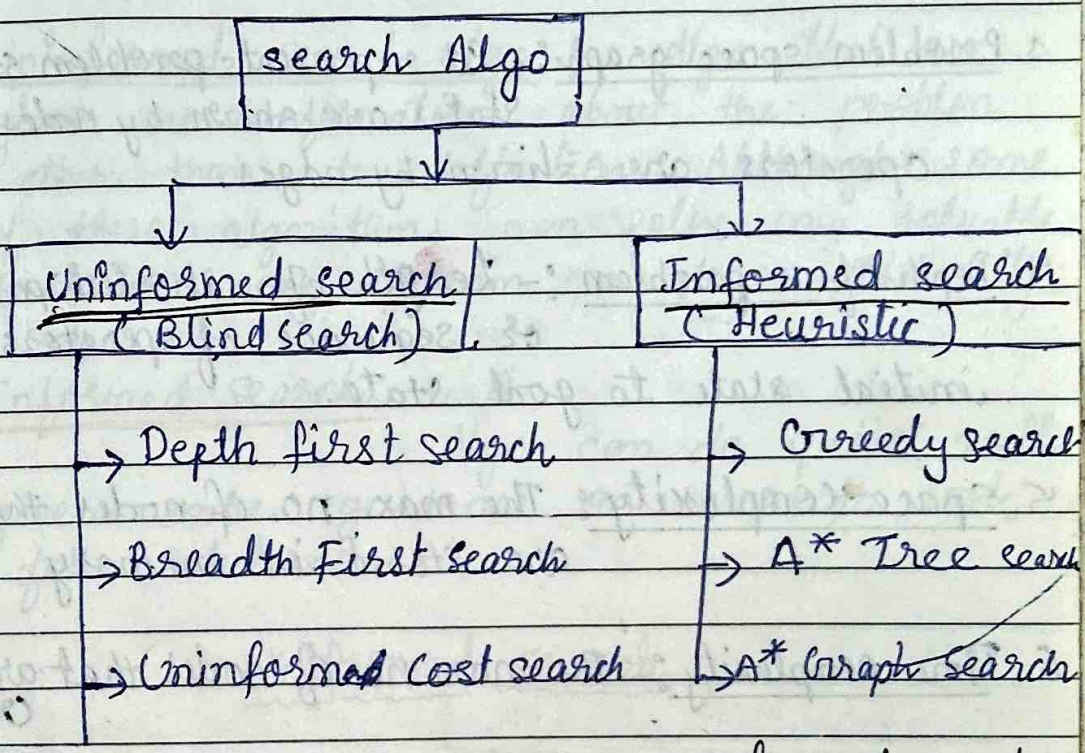
Searching Algo in AI

- Uninformed Search (Blind)
  - Breadth First Search
  - Depth First Search
  - Uniform Cost search
  - Iterative deeping
  - Bidirectional search
- Informed search (Heuristic)
  - Hill Climbing
  - Best First search (Greedy Method)
  - A * search
  - AO * search
  - Creed Search
  - Constraint Satisfaction


Searching Algo in AI
UnInformed search
(Blind search)
Informed search
Heuristic Search
Brute Force search
Depth First search
DFS
Iterative Deepening DFS
Breadth First search
BFS
Bidirectional BFS
Uninformed Cost search
Generate & Test
Hill climbing
Best First seach
(Greedy search)
Simple Hill Cli
Steepest Asce
OR graph
A* Search
Simulated Anne
stochastic hill
AND OR graph
AO* search
* Mean-End-Analysis

Problem solving by SEARCH ALGORITHM

There are far too many powerful search algorithms out there to fit in a single article. Instead, this article will discuss six of the fundamental search algorithms divided into two categories. as shown below



Uninformed search

- Brute force search
- Searching w/o any additional Info
- No domain knowledge
- Time consuming (More complexity (b^d))
- Less efficient
- optimal solution
- Problem states may grow exponentially (NP problems)

Informed Search

- Approximation / Eastimated value
- Given some guidance
- Information about domain
- Quick solution
- More efficient
- optamility May or may not
- Good solu. (Greedy approach)

SEARCH TERMINOLOGY

1. **Problem space :-** It is the environement in which the search takes places.

2. **Problem Instance :-** Initial state + Goal State

3. **Problem space graph :-** It represents problem state. States are shown by nodes & operators are shown by edges.

4. **Depth of a problem :-** Length of a shortest path or sequence of operators from initial state to goal state.

5. **Space Complexity :** The max. no. of nodes that are stored in memory.

6. **Time complexity :-** The max. no. of nodes that are created.

7. **Admissibility :-** A property of an algorithm to always find an optional solution.

8. **Branching Factor :-** The avg. no. of child nodes in the problem space graph

9. **Depth :-** Length of the shortest path from initial state to goal state

↑

"Searching is the universal technique of problem solving in AI. There are some single player such as tile games, Sudoku, crossword etc. The search algorithm help you to search for a particular position in such games."

Unit - 2

Problem solving By Searching

Problem solving agents use atomic representation There are general purpose search algorithms that can be used to solve these problems.

Uninformed search algo :- Algo that are given no information about the problem other than its definition. Although some of these algorithms can solve any solvable problem, none of them can do so efficiently.

Informed search:
Algo can do quite well given some guidance on where to look for solutions.

Uniformed search

Every AI program has to the process of searching are not explicit in nature Basically to do a search process states follows are needed. The initial state description of the problem. Eg the initial of all pieces in chessboard.

→ A set of legal operator that change the rules of the game

→ The final or the goal state - The final position of the problem solver has to reach.

Given this searching can be defined as sequence of steps to the goal state. The ~~seac~~ searching process in AI can broadly classified

1 BRUTE FORCE SEARCH / Uninformed search

These are commonly used search process which explore all the alternatives during the search process they don't have any domain specific knowledge. All they need initial state, the finite state and a set of legal operators.

BFS techniques are -

(1) Depth First Search
(2) Bredth First search

Depth First search :- (Blind Search)

This is very simple type of Brute Force technique. The search begins by expanding the initial node by using an operator generate all successors of the initial node and test ~~ler~~ them.

(instance of Graph searching algo)

Algorithm: BFS Depth first search always expands the deepest node in the current frontier of the search tree.

The search proceeds immediatly to the ~~search~~ deepest level of the search tree, where the nodes have no successors.

As those nodes are expanded, they are dropped from the frontier, so then the search 'backs up' to the next deepest node that still has unexplored successors.

DFS uses a LIFO queue.

A variant of DFS called back tracking search uses still less memory.

Algorithm:

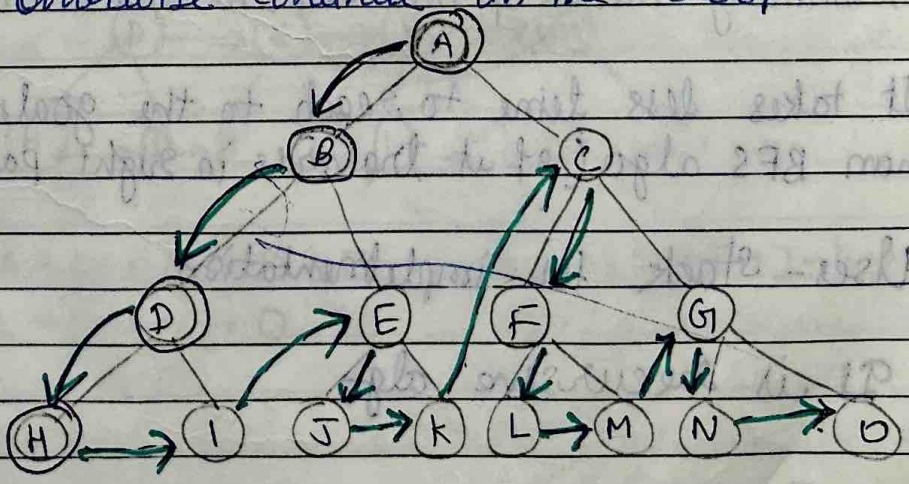
Step 1: If the initial state is a goal state, quite and return sucess

Step 2: Otherwise, do the following until sucess or failure is signaled

(a) Generate a successor, E of the initial State. If there are no more successors, signal failure

(b) Call DFS with E as the initial state.

(c) If sucess is returned, signal sucess. Otherwise continue in the loop.



A → B → D → H → I → E → J → K → C

O ← N ← G ← M ← L ← F

This procedure finds whether the goal can be reached or not, but the path it has to follow, has not been mentioned.

In order to remember the path, all that has to be done is to establish a pointer from each generator nodes.

Disadvantages :-

* Determination of depth until which the search has to be proceed.
→ There is no guarantee of finding the solu^{n}
* This depth is called cut-off depth. The value of cut off depth is essential because other wise the search is going on and on.
* DFS goes for deep down & sometimes it may go to infinite loop.

Advantages

* It requires less memory since only the nodes on the current path are stored
* By chance, DFS may find a solution without examining much of the search space at all.
→ It takes less time to reach to the goal node than BFS algo (if it traverse in right path)
⇒ Uses - stack for implementation
⇒ It is recursive algo

Dep **Breadth First Search (BFS)**

It starts from the root node, explores the neighbouring nodes first and moves towards the next level neighbors. It generates one tree at a time until the solution is found.

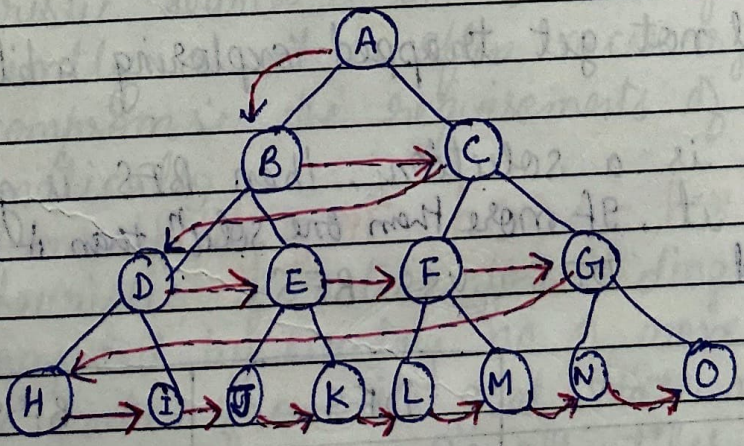
Branching factor = b
(avg. no. child nodes for a given node)

depth = d

no. of nodes at level d = b^d

Disadvantage :-

* It consumes lots of memory space. b/c all of the node that has been generated must be store
* Its complexity depends on the no. of nodes



A → B → C → D → E → F → G → H → I → J → K → L → M → N → O

Algorithm :- BFS

1. Create a variable called NODE-LIST set it to the initial state

2: Until a goal state is found or NODE-LIST empty

(a) Remove the first element from node-list and call it E

(b) For each way that each rule can match the state described in E do:

(i) Apply the rule to generate a new state

(ii) If the new state is a goal state, quite and return this state

(iii) Otherwise, add the new state to the end of NODE-LIST.

Advantages Of BFS

* BFS will not get trapped exploring a blind alley

⇒

* If there is a solution, then BFS is guaranteed to find it. If more than one solu^n then it will provide minimal solu.

DFS VS BFS

Scenario	DFS	BFS
1 Some paths are extremly long or even infinite	Performs badly	Performs well
2. All path are of similar length	Performs well	Performs well
3) All paths are of similar length & leads to a goal state	Performs well	Wastes time and memory

Informed Search :-

The algorithms have information on the goal state, which helps in more efficient seaching. This information is obtained by something called heuristic. It is also called Heuristic search.

Informed search algorithm contains an array of knowledge about ~~search space~~ such as how far we are from the goal, path cost, how to reach to goal node etc. This knowledge help agents to explore less to the search space

Heuristic function :-

A heuristic is a technique that improves the efficiency of a search process, possibly by sacrificing claims of completeness. Heuristic are like tour guides.

* Heuristic search is to solve many hard problems efficiently as it is often necessary to compromise the requirements of mobility and systematically.

Admissibility of HF $h(n) <= h*(n)$

* A heuristic function for sliding tiles games is computed by counting no of moves that each tile makes from its goal state and adding these no. of moves for all tiles.

* It proceeds per preferentially through nodes that heuristic, problem specific informan indicates might be on the best path to a goal. following the basic points of this search method are discussed below:

Heuristic cost should be $\leq$ estimated cost

1. A heuristic evaluation function f is used to help decide which node is the best one to expand next. (f is pronounced as f-hat)

2. Expand next that node n, having the smallest value of f(n).

3 Terminate when the node to be expanded

A* Search :- Greedy
Best First search and A* search

This search consists of an evaluation function variant of breadth first search. The heuristic function is used here is called evaluation function. It is an indicator of how far the node is from goal node. goal node has evaluation function value is zero.

The Greedy best first search algo is implemented by priority queue

* The best first search follows the best path available from the current tree.

* Unlike hill climbing, the best first search provides a scope of corrections, in case a wrong step has been selected earlier. This is the prime advantage of the best first search algorithm over hill climbing.

* It is the combination of BFS & DFS

* At each level/step we select most promising node

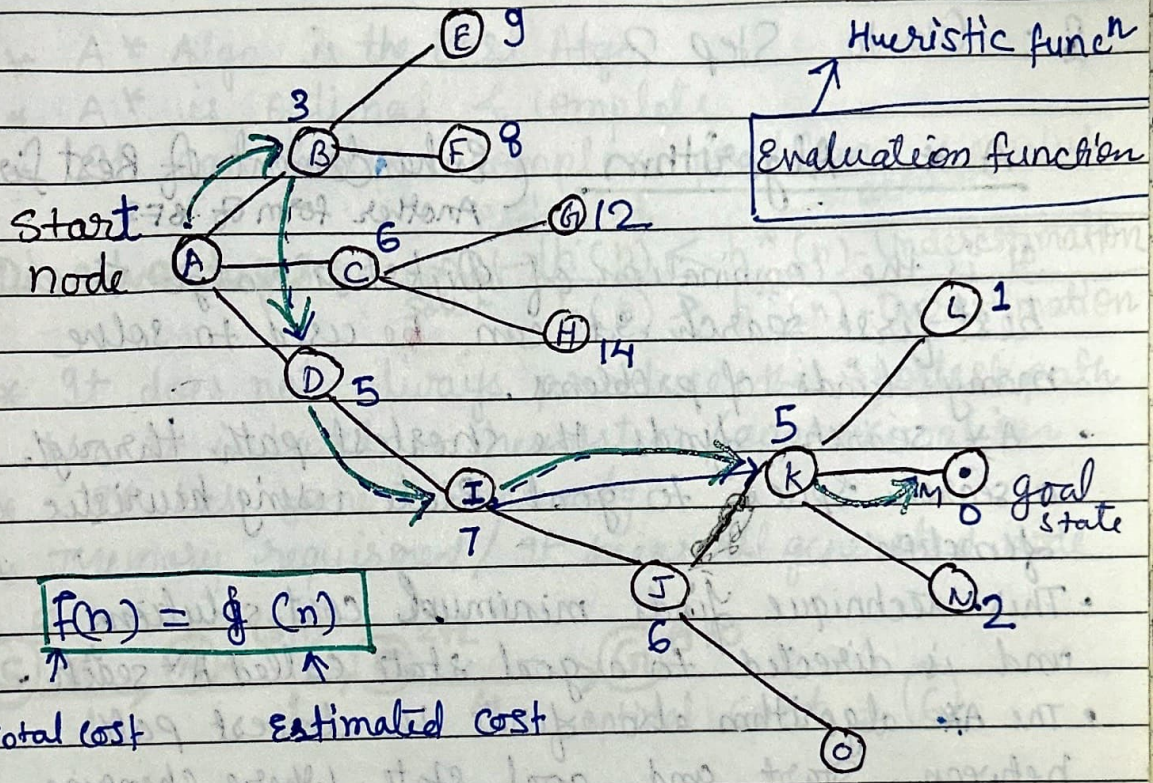
* It is implemented by priority queue

* In avg case / Best case Best first search is better

Evalution value → How far is to goal state from current state

Time & space complexity in worst case → $O(b^m)$

Best First search algorithm.



Step 1: Put the initial node on a list START

2: If START is empty or START = GOAL terminate search

3: Remove the first node from START, call this node 'A'

4: If A' = Goal terminate search with success.

5: Else if node a has a successor generate all of them. Find out how far they are from the goal node. sort all the children generated so far by the remaining distance from the goal.

6. Name this list as START 1

→ It get stuck in a loop as DFS
→ It is not optmal

7: Replace START with START 1

8: Go to Step 2.

A* Algorithm (Enhancement of Best First)
Another form of BFS

It is the combination of Dijkstra's algo & best first search. It can be used to solve many kinds of problems.

- A* search finds the shortest path through a search space to goal state using heuristic function
- This technique finds minimum cost solutions and is directed to a goal state called A* search.
- The A* algorithm also finds the lowest path between start and goal state, where changing from one state to another requires some cost.
- A* requires heuristic function to evaluate the cost of path that passes through the particular state.
- It ~~is~~ avoids expanding paths that are already expensive. but expands most promising path first.

$$f(n) = g(n) + h(n)$$

where

- g(n) the cost (so far) to reach the node from start node
- h(n) estimated cost to get from the node to the goal. (cost to reach from node n to goal node)
- f(n) estimated total cost of path through n to goal. It implemented using priority queue by increasing f(n) (cheapest solution)

"Time complexity depends on the quality of heuristic"

AO* Algo.

Advantages :- of A* (A* Admissible) → (always gives optimal solution)

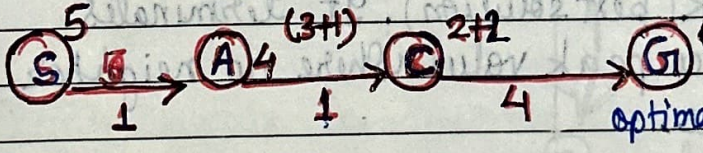
* A* Algo is the best Algo
* A* is optimal & complete
* It can solve complex problem

Disadvantages :- optimal soln → estimated → actual value

$h(n) \leq h^*(n)$ - Underestimation

$h(n) \geq h^*(n)$ - Overestimation

* It does not always produce the shortest path b/c it is based on heuristics / approximation
* It has some complexity
* Memory requirment / It keeps all generated node



At each point in the search, only node is expanded which have the lowest value of $f(n)$.

space complexity → $O(b^d)$ ← Time complexity

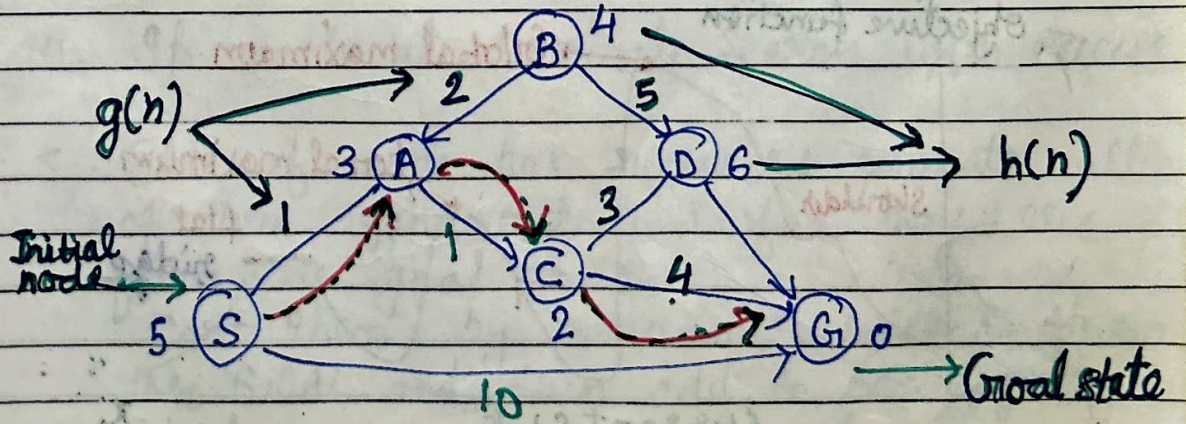
When branching factor is finite & cost of every action is fixed (complete)

It is complete & Admissible

It maintain 2 sets

OPEN SET - set of nodes that needs to be visited

CLOSED SET - set of nodes that have already been visited

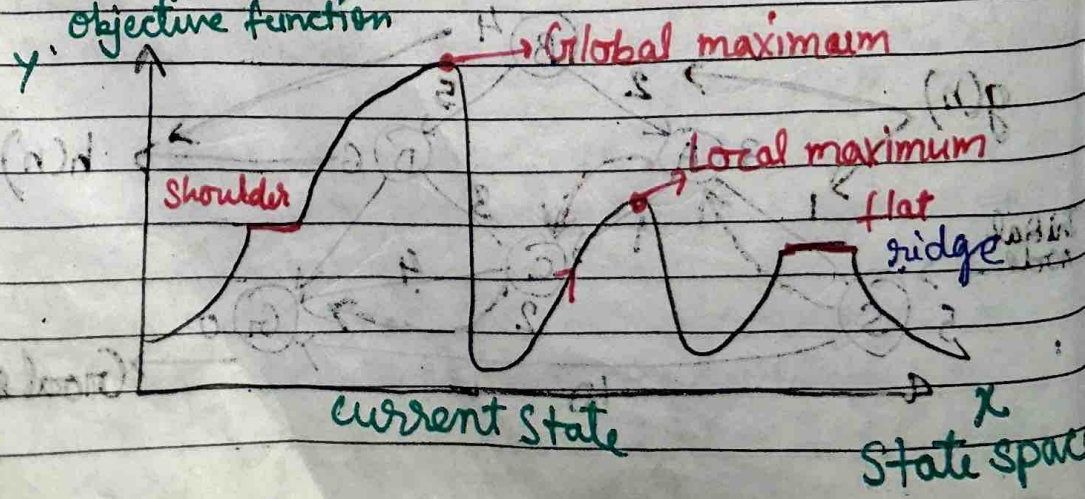


Hill Climbing Search

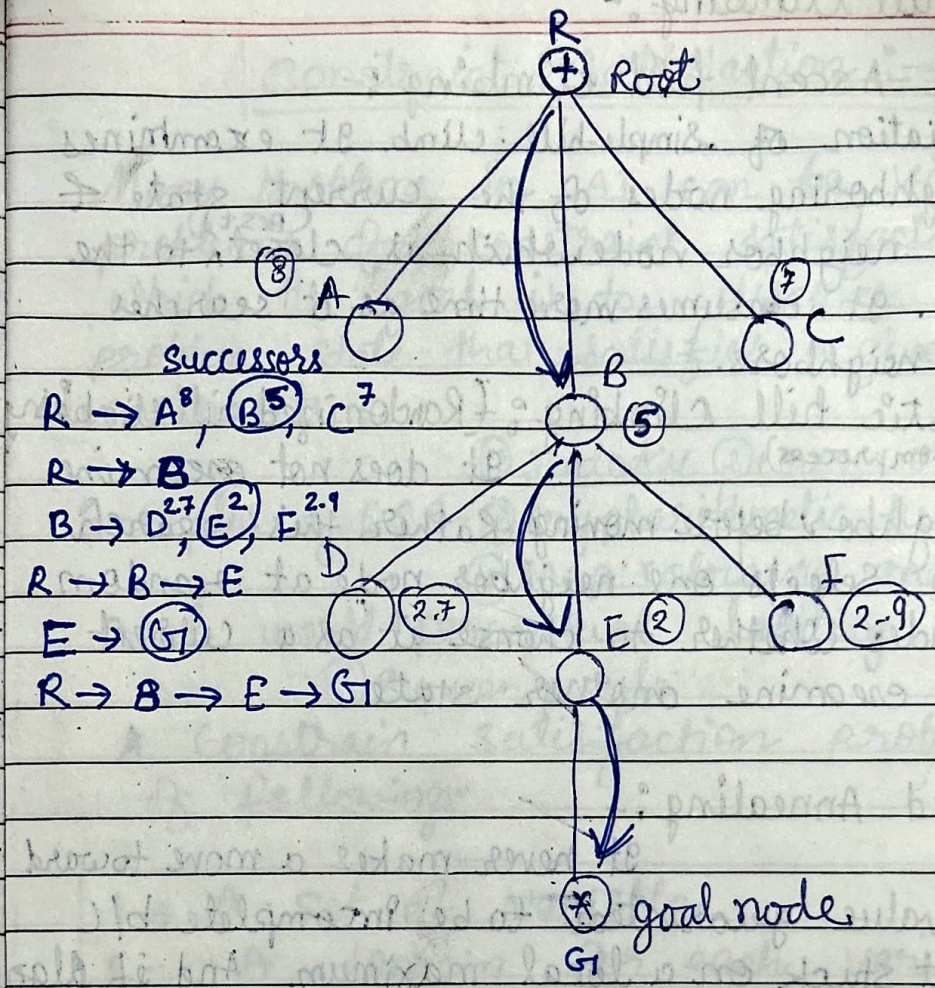
It is an iterative algorithm that ~~state~~ starts with an ~~arbi~~ arbitary solution to a ~~progrem~~ problem and attempts to find a better solution by changing a single element of the solution incrementally. If the change produces a better solution, an incremental change is taken as a new solution. This process is repeated until there are no further improvements.

* It continuously moves in the direction of increasing values to find peak (best solution). It terminates when it reaches a peak value where no neighbor has a higher value.

* It is used for optimizing the mathematical problem
  eg - Travelling salesman problem
* It is also called greedy local search. mostly used when good heuristic is available. (Greedy)
* It only keeps a single current state
* No backtracking
* It is the varient of Generate & Test method.



D



Algorithm :-

1. Put the initial node on a START

2. If START is empty or START = GOAL terminate the search

3. Remove the first node from START call this node a.

4. If a = Goal terminate search with succes.

5. Else if node has sucessors generate all of them. find out how far they are from the goal node.
Sort them by remaining distance from the GOAL and then add to begining of Start.

6. Go to step 2

1. Simple hill climbing :-

2. Steepest-Ascent hill climbing :-
It is variation of Simple-hill climb. It examines all the neighboring nodes of the current state & select one neighbor node which is closest (best) to the goal state. It consumes more time as it searches multiple neighbors.

3. Stochastic hill climbing :- (Random process) {Randomized hill climbing}
It does not examine for all neighbor before moving. Rather this search algorithm selects one neighbor node at random and decides whether to choose it as a current state or examine another state.

4. Simulated Annealing :-
It never makes a move toward a lower value guaranteed to be incomplete b/c it can get stuck on a local maximum. And if algo applies a random walk by moving a successor, then it may complete but not efficient.
* Simulated annealing is an algo. which yields both efficiency and completeness.

---

3. If the set of constraints contains a contradiction then it is reported that this path is dead end.

4. If the set of constraint can describe a complete solution then it is reported as success.

5. If neither a constraints, not a complete solution has been found, then the rules are applied to generate new partial solution these partial solution are inserted into search graph.

Constraint Satisfaction :-

Many problems in AI can be viewed as problems of constraint satisfaction in which the goal is to discover some problem state that satisfies a given set of constraints.

Examples of CSP - ① sudoku ② crossword problem ③ cryptarithmetic puzzles, ④ Map colouring and many real world perceptual labeling problems. ⑤ 8 queen puzzle.

A constrain satisfaction problem consist of following:

1. A set of variable
2. A domain for each variable
3. A set of constraints.

A basic form of the constraints satisfaction procedure is as follows:

1- Until a complete solution is found or until all the paths have led to lead ends. do select an unexpanded node
2. The ~~node~~ constraint inference rules are applied to the selected node to generate all constraints
3. If the set of constraints contains a contradiction then it is reported as success
4. If the set of constraints describe a complete solution then it is reported as success

Constraint satisfaction is a search procedure that operates in a space of constraint sets.

→ The initial state contains the constraints that are originally given in the problem description. A goal state is any state that has been constrained "enough" where 'enough' must be defined for each problem.

⇒ Constraint satisfaction is a two step process.

- First, constraints are discovered and propagated as far as possible throughout the system.
- Then, if there is still not a solution, search begins.
- A guess about something is made and added as a new constraints.

Problem formulation of CSP

- Initial state
- Sucessor function
- Goal test
- Path cost

Criptarithmetic Problem :-

Rules / constraints

1. Digits ranges from 0 to 9 only
2. Each variable should have unique & distinct value.
3. Each letter, symbol represents only one digits throughout the problem.
4. You have to find the value of each other letter in the ~~Eyz~~ cryptarithmetic.
5. There must be only one solution ~~to~~ to the problem.
6. Number must not begin with zero i.e. 0123 (wrong) 123 (correct)
7. After replacing letters by their digits the resulting arithmetic operations must be correct.

Ex.-

```
  SEND
+MORE
------
MONEY
```

	$C_3=0$	$C_2=1$	$C_1=1$	
	9	5	4	3
+	1	0	8	5
1	0	6	5	2

$C_4 = 1$

$C_3 = 0$ or 0

$C_2 = 1$

$M = C_3 + S + 1$

$N = E + 1$

```
  1 1 1
  9 5 6 7
+ 1 0 8 5
---------
1 0 6 5 2
```

$S = 9$

$E = 4$

$N = 5$

$D = 7$

$M = 1$

$O = 0$

$R = 8$

$Y = 3$

IV PROBLEM REDUCTION SEARCH

(Decomposition)

So far, we have considered search strategies for OR graphs through which we want to find a single, path to a goal.

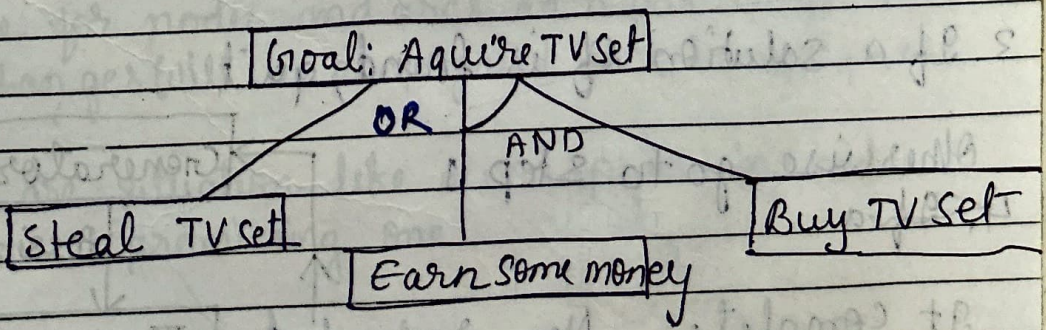
AND-OR Graph

(AO* search) The AND-OR graph is useful for representing the solution of problems that can be solved by decomposing them into a set of smaller problem, all of which must then be solved.

This decomposition, or reduction generates arcs that we call AND arc.

One AND arc may point to any no. of successor node, all of which must be solved in order for the arc to point to a solution.

Just as in an OR graph several arcs may emerge from a single node, indicating a variety of ways in which the original problem might be solved.



A Simple AND-OR graph

D-2018 - 1, 2

Algorithm

Step 1. Initialize the graph to starting node

Step 2. Loop until the starting node is labled solved or until its cost goes above FUTILITY

(a) → Traverse the graph, starting at the initial node and following the current best path & accumulate nodes that r on that path & hv'nt yet been expanded

(b) Pick 1 of these unexpanded nodes & expand it. If there r no successor. assign FUTILITY otherwise add its sucessor to the graph & for each of them compute f. If f of any node is 0 mark that node as solved.

(c) Change the f estimate of the newly expanded node to reflect the new information provided by its sucessor arc whose descendants are all solved. Label the node if itself a SOLVED.

Generate & Test :- (Heuristic) (Basic algo of all Heuristic searching)

(DFS with backtracting)

(Hil and try)

1) Generate a possible solution

(No of states are generated)

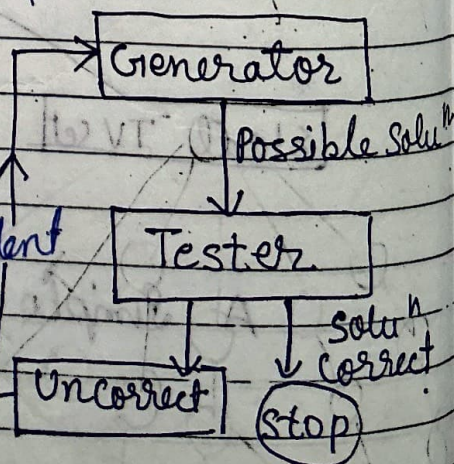
2) Test to see if this a actual solution

3 If a solution is found, quit

Otherwise go to step 1

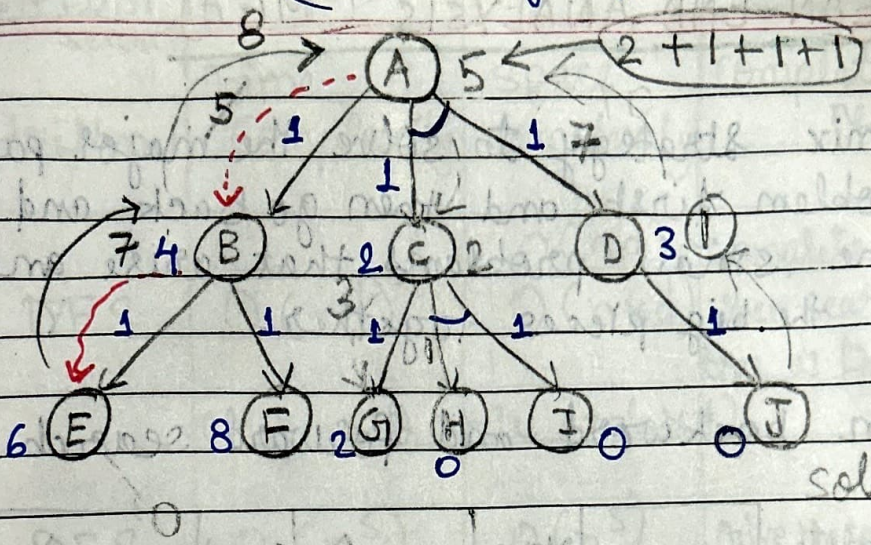
4. If a

It complete, Non redundent



AO* (~~Not~~ Admissible)

(Not always gives optimal solution)



solution graph

→ AO* does not explore all the solution paths once it got a solution.

→ It finds the new heuristic value while exploring the nodes.

→ Always find the minimum cost solution

* AO* algo is the modified ver. of A*. It covers all limitation of A* Algo.

* AO* is always able to find solution with minimum cost.

* It works on 2 phases. 1st phase will find a heuristic value for nodes and arcs in a particular level. The changes will be propogated back in next phase.

* There are situation like a non promising node become a promissing one

SMA* → Simplified memory bounded A*

Algorithm	Time Complexity	Space complexity	Completeness	Optimality
~~Greedy~~ Recursive Best First search	$O(d)$	$O(db)$		may find optimal sol.
1 DFS	$O(n^d)$ ($n \rightarrow$ no of node explored)	$O(n)$ $O(n \times d)$	complete when search tree is finite	Not optimal
2 BFS	$O(n^s)$ $s \rightarrow$ shallowest solution	$O(n^s)$	Give the solu^n if exist	BFS is optimal as long as the cost of all edges are equal
3. Best First search (Greedy)	$O(b^m)$ (in worst case)	$O(b^m)$	Incomplete even if the search space is finite.	Not optimal
4. A*	($m \rightarrow$ maximum depth of search space) $O(b^d)$	$O(b^d)$	complete when B.F. is finite & cost of every step is fixed	Gives optimal Solu
Computational & space complexity is too high.				
5. Hill (simple) Climbing	~~$O(\infty)$~~ $O(d)$	$O(b)$	Not guaranteed	less optimal
Mean End Analysis				
6. AO*	$O(b^d)$	$O(b^{d+1})$	complete	No guarantee less optimal
7. Alpha beta Pruning	$O(b^{m/2})$			
10 Mini Max	$O(b^m)$	$O(bm)$	complete	optimum
8 Branch & Bound.				
9 Uninformed cost search	$O(n^{c/\varepsilon})$ $c/\varepsilon \rightarrow$ effective depth	$O(n^{c/\varepsilon})$	Complete	optimal

Branch and bound Algorithm

* B & B is an generally used for solving combinational optimization problems & mathe-matical optimization
It may requir exploring all possible permutn in worst case.
0/1 knapsack problem can be solved by B & B

* based on DFS with hueristic function.
(Depth bounded search)
* It is applicable when many paths to a goal state exist and we want an optimal path.

* The idea of B & B search is to maintain the lowest-cost path to a goal found so far.
suppose cost is bound

$$\text{cost}(p) + h(p) \geq \text{bound}$$

path p can be primed, if a non pruned path to a goal is found. It must be better than the previous best path.

This new solution is remembered & bound is set to the cost of this new solution.
It then keep searching for better solution.

* B & B keeps track of all partial path which can be candidate for further exploration.

* It saves all path costs from start node to all generated nodes and chooses the successor with shortest cost.

* B & B is feature selection mtd

NLP - Natural Language Processing

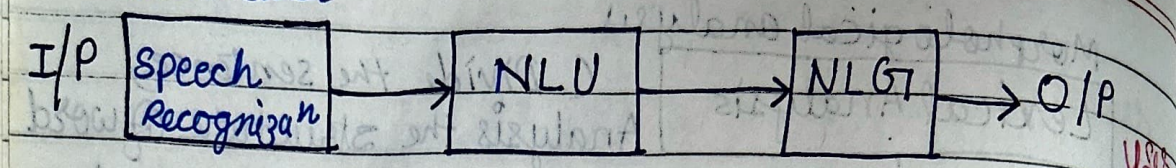
NLP is a subfield of linguistic, comp. Sc., Informa" Engineering and AI concerned with the interaction between computers and human (natural) languages in particular how to program computers to process and analyze large amounts of natural language data.

The ultimate objective of NLP is to read, decipher, understand, and make sense of the human languages in a manner that is valuable.

* Most NLP techniques rely on machine learning to derive meaning from human languages.

* Infact, a typical interaction between humans and machines using NLP could go as follows.

1. A human talks to the machine.
2. The machine captures the audio
3. Audio to text conversion takes place
4. Processing of the text's data.
5. Data to audio conversion takes place
6. The machine responds to the human by playing the audio file.



→ NLU - Natural language understand
(What do you the user say? thier intention & meaning)

Ex-① The tank was full of water.

② The car hit the pole while it was moving

Ambiguity Remove → Lemmanization, Stemming tokenization, parsing.

Police are coming

→ NLGT - (Natural language generator) - Reply
What machine should be say to user
→ Text & sentence planning
→ It should be intelligent and conversational.
→ Deal with structured analysis

Old men and women were taken to Safe place. old ②

Steps Of NLP

Morphological analysis

1. **Lexical Analysis** — (Divide the sentence) Analysis the structure of word (Remove commas, punctuation marks.)

2. **Syntactic Analysis** — Combined the words and create the sentence for grammer

3. **Semantic Analysis** — Meaning of the sentences (text)

4. **Disclosure Integration** — (Integration) Analysis the meaning of word based on previous context

5. **Pragmatic Analysis** — Reinterpreted the sentence for final meaning

Ex - Set alarm at 6.00 PM